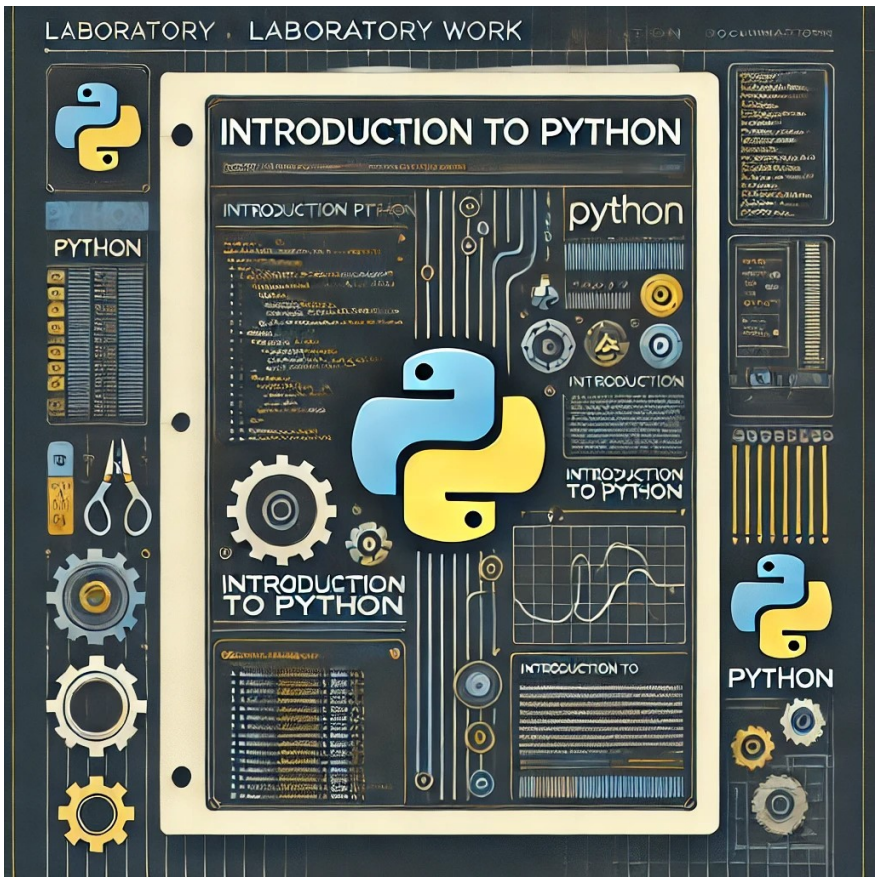
SISTEME BAZATE PE CUNOȘTINȚE

Lucrare de laborator · Anul universitar 2026–2027

Lucrare de laborator 1

Introducere în Python

Python și Jupyter pentru lucrul cu date: instalați, învățați și exersați



CURS	INSTRUMENTE	NOTEBOOK-URI
Sisteme bazate pe cunoștințe	Python 3.10+ · Jupyter	Exemple rezolvate + exerciții

Cuprins

1 Configurarea mediului de lucru 4
1.1 Trei moduri de a rula Jupyter 4
1.2 Instalarea locală, pas cu pas 4
1.3 Primul vostru notebook 5
1.4 Rezolvarea problemelor 6
2 Python în biroul inteligent 7
2.1 Obiecte, tipuri și variabile 7
2.2 Operatori 8
2.3 Containere 8
2.4 Șiruri de caractere și formatare 10
2.5 Structuri de control 11
2.6 Funcții 12
2.7 Clase: propriile obiecte 13
2.8 Funcții predefinite și metode pentru șiruri 15
2.9 Erori și excepții 15
2.10 Fișiere: citire și scriere 15
2.11 Module și importuri 17
2.12 Prima privire asupra instrumentelor pentru date 17
3 Miniproiect: raportul privind calitatea aerului din birou 19
4 Concluzii și fișă recapitulativă 20
5 Temă 21

Acest laborator într-o pagină

DUPĂ ACEST LABORATOR VEȚI PUTEA

- să configurați un **mediu virtual** și să rulați Jupyter local, în VS Code sau în Google Colab;
- să scrieți cod Python cu **variabile, containere, bucle, funcții și clase**;
- să folosiți **f-string-uri**, secționarea (slicing) și **comprehensiunile** așa cum o fac profesioniștii;
- să tratați **erorile** cu `try / except` și să citiți și să scrieți **fișiere** (text, CSV, JSON);
- să prelucrați un jurnal real de senzori din **biroul inteligent** folosit pe parcursul cursului.

CUM SE DESFĂȘOARĂ LABORATORUL

- **Citiți** acest îndrumar: idei, exemple și rezultatul pe care ar trebui să îl obțineți.
- **Rulați** aceleași exemple în `Lab1_Exemple_Rezolvate.ipynb` și modificați-le.
- **Exersați** în `Lab1_Exercitii.ipynb`: 8 probleme, fiecare cu o autoverificare (✅ / ❌).
- **Predați** notebook-ul de exerciții cu toate verificările în verde.
- Lucrați în ritmul vostru; stelușele ★ indică dificultatea.

Fișierele acestui laborator

Fișier	Ce conține
`Lab1_Exemple_Rezolvate.ipynb`	Toate exemplele din acest îndrumar, gata de rulat, cu sarcini „Încercați”
`Lab1_Exercitii.ipynb`	Exercițiile voastre: 8 probleme cu autoverificare automată
`smart_office_dataset.csv`	O săptămână de măsurători, la fiecare minut, din biroul inteligent al cursului

EXEMPLUL DE LUCRU

Exemplele din acest îndrumar folosesc un **birou inteligent**: senzorii măsoară în fiecare minut temperatura, CO_2, umiditatea și numărul de persoane. Aceleași date revin în laboratoarele următoare și pe parcursul cursului, așa că tot ce învățați aici va fi refolosit.

1 Configurarea mediului de lucru

Python este instrumentul acestui curs. Jupyter vă permite să combinați **cod, rezultate și explicații** într-un singur document, numit *notebook*: codul este împărțit în **celule** pe care le rulați pe rând, iar toate celulele unui notebook folosesc aceeași memorie (același *spațiu de nume*, *namespace*). O variabilă definită într-o celulă poate fi folosită în celulele rulate după ea.

> ORDINEA CONTEAZĂ
> Rulați celulele **în ordine**. Dacă o celulă folosește o variabilă definită mai jos, apare o eroare `NameError`. Când aveți dubii, folosiți *Kernel → Restart & Run All*.

1.1 Trei moduri de a rula Jupyter

Variantă	Instalare?	Potrivită pentru	Legătură
Local (Anaconda sau Python + pip)	da	Lucru zilnic; datele rămân pe calculatorul vostru	anaconda.com · python.org
Google Colab	nu	Start rapid, GPU gratuit, funcționează din orice browser	colab.research.google.com
VS Code cu extensia Jupyter	da	Notebook-uri, scripturi și Git într-un singur editor	code.visualstudio.com
JupyterLite (în browser)	nu	O încercare de un minut; experimental, nu se salvează nimic	jupyter.org/try

Tot ce facem la acest curs funcționează în toate variantele. JupyterLab este interfața actuală; Notebook-ul clasic (versiunea 7) folosește același motor. Documentație: jupyter.org și docs.python.org.

1.2 Instalarea locală, pas cu pas

- **Instalați Python 3.10 sau mai nou** (versiunea stabilă curentă este 3.14) de pe python.org – sau instalați Anaconda, care conține deja Jupyter.
- **Creați un mediu virtual** pentru curs. El ține pachetele acestui curs separate de tot restul din calculatorul vostru.
- **Activați-l**, apoi **instalați pachetele** pe care le folosim.
- **Porniți Jupyter** și creați primul notebook.

TERMINAL

```
python -m venv .venv                  # 1. creați mediul (o singură dată)
.venv\Scripts\activate                # 2. activați-l - Windows
source .venv/bin/activate             #    activați-l - Linux / macOS
pip install notebook jupyterlab numpy scipy pandas matplotlib seaborn scikit-learn
pip install statsmodels               # 3. instalați pachetele cursului
jupyter lab                           # 4. porniți Jupyter (sau: jupyter notebook)
```

SFAT

În interiorul unui notebook instalați pachetele cu `%pip install <pachet>`, **nu** cu `!pip`. Comanda `%pip` instalează în *același* mediu cu kernel-ul care rulează, deci importul funcționează imediat (reporniți kernel-ul dacă pachetul fusese deja importat).

Acestea sunt pachetele folosite la curs: numpy, scipy, pandas, matplotlib, seaborn, scikit-learn, statsmodels. Pachetele suplimentare (de exemplu `fg-data-profiling`, `sweetviz`) sunt introduse când este nevoie.

DACĂ O INSTALARE EȘUEAZĂ

Unele pachete nu sunt încă disponibile pentru cea mai nouă versiune de Python. Dacă o instalare eșuează, creați mediul cu o versiune mai veche de Python – consultați documentul *How to create a virtual environment with a different version of Python* din dosarul laboratorului.

1.3 Primul vostru notebook

- În Jupyter alegeți **File → New → Notebook**.
- Selectați kernel-ul **Python 3 (ipykernel)**.
- Tastați `print("Hello world!")` în prima celulă și apăsați **Shift + Enter**.

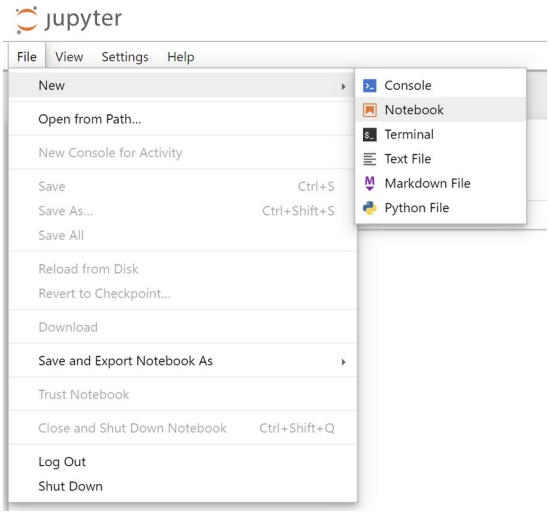


Figura 1. File → New → Notebook

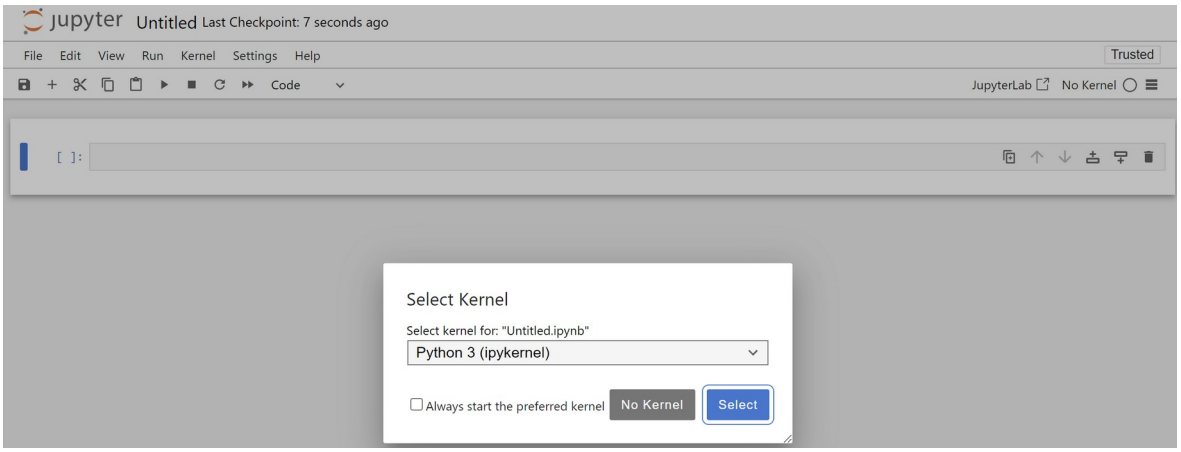


Figura 2. Alegeți kernel-ul Python 3; prima celulă este pregătită

Acțiune	Cum
Rulați o celulă și treceți la următoarea	`Shift + Enter`
Inserați o celulă deasupra / dedesubt	`A` / `B` în modul de comandă, sau butonul +
Ștergeți o celulă	`D D` în modul de comandă, sau *Edit* → *Delete Cells*
Transformați o celulă în text (Markdown)	`M` în modul de comandă (înapoi la cod: `Y`)
Reporniți și rulați totul	*Kernel* → *Restart Kernel and Run All Cells*
Salvare / descărcare	`Ctrl + S`; notebook-urile se salvează în dosarul din care ați pornit Jupyter. *File* → *Download* vă dă fișierul `.ipynb`

Celulele de text folosesc **Markdown** (markdownguide.org): `# Titlu`, `**aldin**`, liste, legături și formule.

1.4 Rezolvarea problemelor

`error: Cargo, the Rust package manager, is not installed`

Unele pachete trebuie compilate atunci când nu există o versiune gata făcută pentru versiunea voastră de Python. Mesajul `Preparing metadata (pyproject.toml) did not run successfully … Cargo is not installed` înseamnă că lipsește Rust. Instalați-l de la rustup.rs (pe Windows poate fi nevoie și de instrumentele de compilare Visual Studio) și repetați instalarea. Adesea un remediu mai simplu este să folosiți o versiune de **Python puțin mai veche**, pentru care există pachete gata compilate.

`'jupyter-notebook' is not recognized as an internal or external command`

Windows nu știe unde este instalat Jupyter: dosarul său nu se află în variabila de mediu **PATH**.

- Găsiți dosarul, de obicei `C:\Users\<voi>\AppData\Local\Programs\Python\PythonXX\Scripts\` (căutați `jupyter.exe`).
- Deschideți *Start*, scrieți „Environment Variables” și alegeți **Edit the system environment variables → Environment Variables**.
- La *System variables* selectați **Path → Edit → New**, lipiți dosarul și confirmați cu OK peste tot.
- Deschideți un terminal **nou** și tastați `jupyter notebook`. (Dacă PATH nu este problema, puteți folosi și `python -m notebook` sau `python -m jupyterlab`.)

2 Python în biroul inteligent

Python este un limbaj de nivel înalt, de uz general, interpretat, care pune accent pe lizibilitate: **indentarea** definește blocurile și nu există simboluri de sfârșit de instrucțiune. Tipurile sunt determinate dinamic, memoria este gestionată automat, iar limbajul susține stilurile orientat pe obiecte, funcțional și procedural. Datorită simplității și ecosistemului său uriaș, este limbajul standard al științei datelor și al învățării automate. Referința completă: docs.python.org.

În secțiunile de mai jos fiecare concept este ilustrat cu **biroul inteligent**. Tot codul se găsește în `Lab1_Exemple_Rezolvate.ipynb`.

2.1 Obiecte, tipuri și variabile

În Python **totul este un obiect**, iar o variabilă este doar un *nume* atașat unui obiect. Nu declarați niciodată un tip; Python îl deduce. Tipurile de bază sunt `int`, `float`, `str` și `bool`.

```
temperature = 21.4        # float  (°C)
occupancy   = 7           # int    (persoane în încăpere)
room        = "Lab-204"   # str
window_open = False       # bool

for value in (temperature, occupancy, room, window_open):
    print(f"{value!r:>10}  ->  {type(value).__name__}")
```

REZULTAT

```
      21.4  ->  float
         7  ->  int
 'Lab-204'  ->  str
     False  ->  bool
```

Python este **tipizat dinamic** (același nume se poate referi mai târziu la un alt tip), dar și **tipizat puternic**: operațiile fără sens produc o eroare în loc să ghicească.

```
try:
    print(temperature + room)
except TypeError as err:
    print("TypeError:", err)
```

REZULTAT

```
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

Convenția de denumire (PEP 8, peps.python.org/pep-0008): cuvinte scrise cu litere mici, separate prin underscore (`co2_ppm`, `max_temperature`); numele pot conține litere, cifre și `_` și nu pot începe cu o cifră.

2.2 Operatori

Grup	Operatori
Aritmetici	`+ - * / // % **` (împărțire reală, împărțire întreagă, rest, putere)
De atribuire	`= += -= *= /=`
De comparare (dau `True` / `False`)	`== != < <= > >=`, înlănțuiți: `20 <= t <= 24`
Logici	`and or not`

```
t_c = 21.4
t_f = t_c * 9/5 + 32                          # Celsius -> Fahrenheit
print(f"{t_c} °C = {t_f:.1f} °F")
print(7 / 2, 7 // 2, 7 % 2, 2 ** 10)

co2 = 1180
too_high    = co2 > 1000
comfortable = 20 <= t_c <= 24
print("CO2 prea mare:", too_high, "| confortabil:", comfortable)
print("Deschideți fereastra:", too_high and not comfortable)
```

REZULTAT

```
21.4 °C = 70.5 °F
3.5 3 1 1024
CO2 prea mare: True | confortabil: True
Deschideți fereastra: False
```

Parantezele schimbă precedența operatorilor (tabelul complet). În Jupyter, valoarea **ultimei expresii** dintr-o celulă este afișată automat; un `;` la sfârșit o ascunde, iar `print()` afișează orice altceva.

SURPRIZA VIRGULEI MOBILE

Numerele în virgulă mobilă sunt stocate în binar, deci unele zecimale sunt inexacte. Comparați numerele `float` cu `math.isclose`, niciodată cu `==`.

```
import math
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3))
```

REZULTAT

```
0.30000000000000004
False
True
```

2.3 Containere

Un container grupează alte obiecte. Șirurile de caractere sunt containere de caractere. Cele patru containere de bază diferă prin patru proprietăți:

Tip	Exemplu	Ordonat	Mutabil	Duplicate	Utilizare tipică
`list`	`[21.4, 21.6]`	da	da	da	o serie de măsurători
`tuple`	`(46.77, 23.60)`	da	nu	da	înregistrare fixă, coordonate
`set`	`{"A", "B"}`	nu	da	nu	elemente unice, test de apartenență
`dict`	`{"temp": 21.4}`	ordinea inserării	da	chei unice	câmpuri denumite, tabele de căutare

DICȚIONARELE SUNT ACUM ORDONATE

Îndrumarele mai vechi spun că dicționarele nu sunt ordonate. Acest lucru era adevărat până la Python 3.5. **Începând cu Python 3.7 un dicționar își păstrează ordinea inserării.** Elementele unei mulțimi trebuie să fie *hashable* (obiecte imutabile, precum numere, șiruri, tuple). Obiectele mutabile pot fi modificate după creare; cele imutabile nu.

Liste, indexare și secționare (slicing)

```
temps = [19.1, 19.0, 19.4, 20.2, 21.7, 22.5, 23.1, 22.8]     # la fiecare 10 minute
print("primul:", temps[0], "| ultimul:", temps[-1])
print("secțiunea [2:5]:", temps[2:5])        # start inclus, stop exclus
print("din 2 în 2   :", temps[::2])
print("inversat     :", temps[::-1])
temps.append(22.1)                          # listele sunt mutabile
print(len(temps), min(temps), max(temps), round(sum(temps) / len(temps), 2))
```

REZULTAT

```
primul: 19.1 | ultimul: 22.8
secțiunea [2:5]: [19.4, 20.2, 21.7]
din 2 în 2   : [19.1, 19.4, 21.7, 23.1]
inversat     : [22.8, 23.1, 22.5, 21.7, 20.2, 19.4, 19.0, 19.1]
9 19.0 23.1 21.1
```

Tuple, mulțimi și dicționare

```
location = (46.77, 23.60)                   # tuplă: imutabilă
rooms = {"Lab-204", "Lab-205", "Lab-204"}   # mulțime: duplicatele dispar
print(len(rooms), "Lab-205" in rooms)

sensor = {"id": "S-17", "type": "CO2", "unit": "ppm", "room": "Lab-204"}
print(sensor["type"], sensor.get("battery", "necunoscut"))   # .get evită o KeyError
sensor["battery"] = 87
print(list(sensor.items())[:2])
```

REZULTAT

```
2 True
CO2 necunoscut
[('id', 'S-17'), ('type', 'CO2')]
```

Comprehensiuni, enumerate și zip

O **comprehensiune** construiește un container nou dintr-unul vechi într-o singură linie lizibilă. O veți folosi constant când lucrați cu date.

```
temps_f = [round(t * 9/5 + 32, 1) for t in temps]                  # transformare
warm    = [t for t in temps if t > 21]                              # filtrare
labels  = {t: ("cald" if t > 21 else "ok") for t in temps[:4]}      # dict comprehension
print(temps_f[:3]); print(warm); print(labels)

for i, (t, h) in enumerate(zip(temps[:3], [47.8, 47.9, 48.1])):    # indice + două liste
    print(f"măsurătoarea {i}: {t} °C, {h} %")
```

REZULTAT

```
[66.4, 66.2, 66.9]
[21.7, 22.5, 23.1, 22.8, 22.1]
{19.1: 'ok', 19.0: 'ok', 19.4: 'ok', 20.2: 'ok'}
măsurătoarea 0: 19.1 °C, 47.8 %
măsurătoarea 1: 19.0 °C, 47.9 %
măsurătoarea 2: 19.4 °C, 48.1 %
```

ÎNCERCAȚI

Creați `cold = [t for t in temps if t < 19.5]`. Câte măsurători conține? Folosiți o secțiune (slice) pentru a obține **ultimele trei** măsurători.

2.4 Șiruri de caractere și formatare

Șirurile sunt secvențe imutabile de caractere. Modul modern de a construi un text este **f-string-ul**: puneți `f` înaintea ghilimelelor și scrieți expresiile între acolade, cu un *specificator de format* opțional după două puncte. Vechiul stil `%` și `str.format()` încă funcționează și apar în cod mai vechi.

```
name, age = "Maria", 21
print("Bună! Mă numesc %s și am %d ani." % (name, age))      # stil vechi
print("Bună! Mă numesc {} și am {} ani.".format(name, age))  # str.format
print(f"Bună! Mă numesc {name} și am {age} ani.")             # f-string (recomandat)

value = 1234.5678
print(f"{value:.2f} | {value:10.1f} | {value:,.0f} | {0.4567:.1%}")
```

REZULTAT

```
Bună! Mă numesc Maria și am 21 ani.
Bună! Mă numesc Maria și am 21 ani.
Bună! Mă numesc Maria și am 21 ani.
1234.57 |     1234.6 | 1,235 | 45.7%
```

Specificator	Semnificație	Exemplu → rezultat
`:.2f`	număr real cu 2 zecimale	`f"{3.14159:.2f}"` → `3.14`
`:>8` / `:<8` / `:^8`	aliniere la dreapta / stânga / centrat pe 8 caractere	`f"{'ab':>5}"` → `   ab`
`:,`	separator de mii	`f"{1234567:,}"` → `1,234,567`
`:.1%`	procent	`f"{0.456:.1%}"` → `45.6%`
`!r`	afișează `repr` al valorii	`f"{'a'!r}"` → `'a'`

```
s = "tHis is a sTriNg"
print(s.capitalize(), "|", s.upper(), "|", s.lower(), "|", s.title())
print(s.count("t"), s.lower().count("t"), s.startswith("tH"), s.replace("is", "IS"))
print("a,b,c".split(","), "-".join(["2026", "03", "02"]), "  padded  ".strip())
```

REZULTAT

```
This is a string | THIS IS A STRING | this is a string | This Is A String
1 2 True tHIS IS a sTriNg
['a', 'b', 'c'] 2026-03-02 padded
```

2.5 Structuri de control

`if / elif / else` ia decizii; `while` repetă cât timp o condiție este adevărată; `for` parcurge **elementele unui container** (o buclă *for-each*, nu o buclă cu contor ca în C). `break` părăsește bucla, `continue` sare la iterația următoare, iar `pass` nu face nimic (un înlocuitor pentru cod pe care îl veți scrie mai târziu). O buclă poate avea o ramură `else` care se execută numai dacă bucla s-a încheiat **fără** `break`.

O GREȘEALĂ CLASICĂ

`range(a, b)` **exclude** b. `range(5)` produce 0, 1, 2, 3, 4 (nu și 5).

```
print(list(range(5)))
print(list(range(2, 8)))
print(list(range(2, 10, 3)))
print(list(range(10, 0, -2)))
```

REZULTAT

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[2, 5, 8]
[10, 8, 6, 4, 2]
```

```
co2_series = [420, 455, 610, 890, 1050, 1320, 980, 760]
for value in co2_series:
    if value < 600:
        level = "bun"
    elif value < 1000:
        level = "acceptabil"
    else:
        level = "ventilați!"
    print(f"{value:5d} ppm -> {level}")
```

REZULTAT

```
  420 ppm -> bun
  455 ppm -> bun
  610 ppm -> acceptabil
  890 ppm -> acceptabil
 1050 ppm -> ventilați!
 1320 ppm -> ventilați!
  980 ppm -> acceptabil
  760 ppm -> acceptabil
```

```
for minute, value in enumerate(co2_series):
    if value is None:
        continue                        # sărim peste valorile lipsă
    if value >= 1000:
        print(f"limită depășită la minutul {minute} ({value} ppm)")
        break
else:
    print("limita nu a fost depășită niciodată")      # doar dacă nu s-a ieșit cu
break

co2, steps = 1320.0, 0
while co2 >= 600:                       # fiecare pas de ventilare elimină 15 %
    co2 *= 0.85
    steps += 1
print(f"{steps} pași -> {co2:.0f} ppm")
```

REZULTAT

```
limită depășită la minutul 4 (1050 ppm)
5 pași -> 586 ppm
```

O expresie *ternară* este un `if` compact: `level = "ridicat" if co2 > 1000 else "normal"`. Atenție la **buclele infinite**: variabila testată de `while` trebuie să se modifice în interiorul buclei.

ÎNCERCAȚI

Schimbați limita `1000` în `1300`. Ce se afișează? Apoi modificați bucla astfel încât să numere **câte** măsurători depășesc limita.

2.6 Funcții

O funcție împachetează o logică: **intrări → instrucțiuni → ieșiri**. Se definește cu `def`; prima ei instrucțiune poate fi un *docstring* care o documentează. Argumentele sunt **poziționale** (identificate prin ordine) sau **cu nume** (identificate prin nume, cu valori implicite). Argumentele poziționale vin întotdeauna primele. Dacă o funcție returnează mai multe valori, acestea formează o tuplă care poate fi despachetată.

```python
def to_fahrenheit(celsius: float) -> float:
    """Convertește o temperatură din Celsius în Fahrenheit."""
    return celsius * 9/5 + 32

def describe(readings, unit="°C", digits=1):
    """Returnează (minim, maxim, medie) pentru o listă de măsurători."""
    mean = sum(readings) / len(readings)
    return min(readings), max(readings), round(mean, digits)

print(to_fahrenheit(21.4))
lo, hi, avg = describe(temps)                 # despachetarea tuplei
print(f"min={lo} max={hi} medie={avg}")
print(describe(temps, digits=3))              # argument cu nume
print(describe(readings=temps, unit="°F"))    # toate cu nume - în orice ordine
lo_only, _, _ = describe(temps)               # păstrăm doar prima valoare
print(lo_only)
```

REZULTAT

```
70.52
min=19.0 max=23.1 medie=21.1
(19.0, 23.1, 21.1)
(19.0, 23.1, 21.1)
19.0
```

Funcțiile sunt obiecte, deci pot fi stocate și transmise mai departe. Un `lambda` este o funcție anonimă minusculă.

```python
convert = {"C->F": lambda c: c * 9/5 + 32, "C->K": lambda c: c + 273.15}
for name, f in convert.items():
    print(name, round(f(21.4), 2))
print(sorted(sensor.items(), key=lambda kv: kv[0])[:2])
```

REZULTAT

```
C->F 70.52
C->K 294.55
[('battery', 87), ('id', 'S-17')]
```

NOTĂ

O funcție `print_func()` fără `return` dă înapoi `None`: `print(print_func())` afișează mai întâi textul funcției și apoi `None`.

2.7 Clase: propriile obiecte

O **clasă** reunește *date* (atribute) și *comportament* (metode). `__init__` este constructorul; `self` este obiectul însuși; `__repr__` controlează cum este afișat obiectul; **moștenirea** permite unei clase să refolosească o alta. Atributele care nu sunt apelabile sunt *proprietăți*; cele apelabile sunt *metode*; `dir(obj)` le listează pe toate.

```python
class Sensor:
    """Un senzor generic, cu nume, unitate de măsură și listă de măsurători."""
    def __init__(self, name, unit, valid_range=(-1e9, 1e9)):
        self.name = name
        self.unit = unit
```

```
        self.valid_range = valid_range
        self.readings = []

    def add(self, value):
        lo, hi = self.valid_range
        if lo <= value <= hi:
            self.readings.append(value)
            return True
        return False                        # valoare în afara intervalului: respinsă

    def mean(self):
        return sum(self.readings) / len(self.readings)

    def __repr__(self):
        return f"Sensor({self.name!r}, {len(self.readings)} valori,
unit={self.unit})"

class Co2Sensor(Sensor):                    # moștenire
    def __init__(self, name):
        super().__init__(name, "ppm", valid_range=(300, 5000))
    def air_quality(self):
        m = self.mean()
        return "bună" if m < 600 else "acceptabilă" if m < 1000 else "slabă"

s = Co2Sensor("co2-lab204")
for v in [420, 455, 610, 150, 890, 2400]:
    print(v, "acceptat" if s.add(v) else "RESPINS")
print(s, "| medie =", round(s.mean(), 1), "|", s.air_quality())
```

REZULTAT

```
420 acceptat
455 acceptat
610 acceptat
150 RESPINS
890 acceptat
2400 acceptat
Sensor('co2-lab204', 5 valori, unit=ppm) | medie = 955.0 | acceptabilă
```

Pentru clasele care doar rețin date, o **dataclass** elimină codul repetitiv (constructorul și `__repr__` sunt generate). Conversiile de tip se fac cu constructorii de tip `int()`, `float()`, `str()`, `list()`, `tuple()`, `set()`.

```
from dataclasses import dataclass

@dataclass
class Reading:
    timestamp: str
    temp_c: float
    co2_ppm: float

r = Reading("2026-03-02 08:00", 19.4, 455.0)
print(r, "|", r.temp_c, "|", int("12") + 1, "|", list("abc"), "|", set([1, 1, 2]))
```

REZULTAT

```
Reading(timestamp='2026-03-02 08:00', temp_c=19.4, co2_ppm=455.0) | 19.4 | 13 | ['a', 'b',
…
```

2.8 Funcții predefinite și metode pentru șiruri

Funcție	Ce face	Exemplu
`type(x)`	tipul unui obiect	`type("Hi")` → `str`
`len(c)`	numărul de elemente	`len([1, 2, 3])` → `3`
`sorted(c)`	listă nouă, sortată	`sorted([3, 1, 2])` → `[1, 2, 3]`
`sum(c)`, `min(c)`, `max(c)`	agregări	`sum([1, 2, 3])` → `6`
`abs(x)`, `round(x, n)`	valoare absolută, rotunjire	`round(2.567, 1)` → `2.6`
`callable(x)`, `repr(x)`, `dir(x)`	introspecție	`callable(len)` → `True`

Metode folosite frecvent pentru șiruri: `.capitalize()`, `.upper()`, `.lower()`, `.title()`, `.strip()`, `.split()`, `.join()`, `.replace(old, new)`, `.count(sub)`, `.startswith(sub)`, `.endswith(sub)`. Lista completă: docs.python.org/3/library/functions.html.

2.9 Erori și excepții

Datele reale sunt „murdare”. În loc să lăsați programul să se oprească, **interceptați eroarea specifică** pe care o așteptați (`ValueError`, `KeyError`, `ZeroDivisionError`, …) și decideți ce faceți cu elementul greșit. Forma completă este `try / except / else / finally`.

```
raw = ["19.1", "19.0", "", "n/a", "19.4", "-5", "20.2"]
clean, bad = [], []
for item in raw:
    try:
        value = float(item)              # poate produce ValueError
    except ValueError:
        bad.append(item)                 # păstrăm o urmă a ceea ce a fost greșit
        continue
    clean.append(value)
print("valide :", clean)
print("greșite:", bad)
```

REZULTAT

```
valide : [19.1, 19.0, 19.4, -5.0, 20.2]
greșite: ['', 'n/a']
```

REȚINEȚI

`-5` a fost convertit fără nicio eroare, dar **−5 °C într-un birou este un cod de eroare al senzorului**. A intercepta o excepție nu înseamnă a *valida* datele. Descoperirea unor asemenea valori este subiectul cursurilor de înțelegere a datelor.

2.10 Fișiere: citire și scriere

Folosiți `with open(...)`: fișierul este închis automat, chiar dacă apare o eroare. Moduri: `'r'` citire (implicit), `'w'` scriere (**șterge** un fișier existent), `'a'` adăugare la sfârșit; adăugați `'b'` pentru fișiere binare. `f.read()` returnează tot textul, `f.readlines()` o listă de linii, parcurgerea lui `f` merge linie cu linie, iar `f.write(s)`

scrie un șir. Pentru căi folosiți `pathlib.Path`; pentru tabele modulul `csv` (sau pandas în laboratorul următor), iar pentru configurări modulul `json`.

```
from pathlib import Path
import csv

path = Path("smart_office_dataset.csv")
print("există:", path.exists(), "| dimensiune:", path.stat().st_size // 1024, "KB")
with path.open(newline="") as f:
    rows = list(csv.DictReader(f))          # fiecare rând devine un dict
print(len(rows), "rânduri; coloane:", list(rows[0].keys()))
print(rows[0])
```

REZULTAT

```
există: True | dimensiune: 437 KB
10080 rânduri; coloane: ['timestamp', 'temp_c', 'humidity_pct', 'co2_ppm', 'occupancy',
'te…
{'timestamp': '2026-03-02 00:00:00', 'temp_c': '19.14', 'humidity_pct': '47.8', 'co2_ppm':
…
```

```
missing, impossible = 0, 0
for row in rows:
    if row["co2_ppm"] == "":
        missing += 1
    elif not 300 <= float(row["co2_ppm"]) <= 5000:    # CO2 < 300 ppm: imposibil
        impossible += 1
print(f"măsurători CO2 lipsă: {missing} | imposibile: {impossible}")
```

REZULTAT

```
măsurători CO2 lipsă: 31 | imposibile: 2
```

```
report = Path("office_report.txt")
with report.open("w", encoding="utf-8") as f:         # 'w' creează / suprascrie
    f.write("Biroul inteligent - audit CO2\n")
    f.write(f"rânduri citite : {len(rows)}\n")
    f.write(f"valori lipsă   : {missing}\n")
with report.open(encoding="utf-8") as f:
    for line in f:
        print(line, end="")                           # end="" evită liniile goale

import json
Path("config.json").write_text(json.dumps({"warn": 1000, "alarm": 1500, "rooms":
["Lab-204"]}, indent=2))
print(json.loads(Path("config.json").read_text())["alarm"])
```

REZULTAT

```
Biroul inteligent - audit CO2
rânduri citite : 10080
valori lipsă   : 31
1500
```

2.11 Module și importuri

Un *modul* este un fișier cu cod reutilizabil. `import math` · `import math as m` (alias) · `from math import sqrt` (doar un nume). Evitați `from module import *`: ascunde de unde provin numele și le poate suprascrie pe ale voastre. Biblioteca standard ajută deja la multe sarcini:

```
import statistics, random, datetime as dt
from collections import Counter

random.seed(42)                                  # aleatoriu reproductibil
sample = [round(random.gauss(21, 1.2), 1) for _ in range(10)]
print(sample)
print("medie", round(statistics.mean(sample), 2), "| mediană",
statistics.median(sample))
start = dt.datetime(2026, 3, 2, 8, 0)
print((start + dt.timedelta(minutes=90)).strftime("%A %H:%M"))
print(Counter(["bun", "bun", "slab", "bun", "acceptabil"]).most_common(1))
```

REZULTAT

```
[20.8, 20.8, 20.9, 21.8, 20.8, 19.2, 21.4, 20.7, 20.7, 21.1]
medie 20.82 | mediană 20.8
Monday 09:30
[('bun', 3)]
```

2.12 Prima privire asupra instrumentelor pentru date

Din laboratorul următor folosim **pandas** (tabele), **numpy** (tablouri rapide) și **matplotlib / seaborn** (grafice). Un prim exemplu: o singură linie înlocuiește bucla pe fișier din secțiunea 2.10.

```
import pandas as pd
df = pd.read_csv("smart_office_dataset.csv", parse_dates=["timestamp"],
index_col="timestamp")
print(df.shape)
print(df["co2_ppm"].describe().round(1))
```

REZULTAT

```
(10080, 5)
count    10049.0
mean       497.7
std        113.4
min        150.0
25%        420.0
50%        428.0
75%        602.0
max       2400.0
Name: co2_ppm, dtype: float64
```

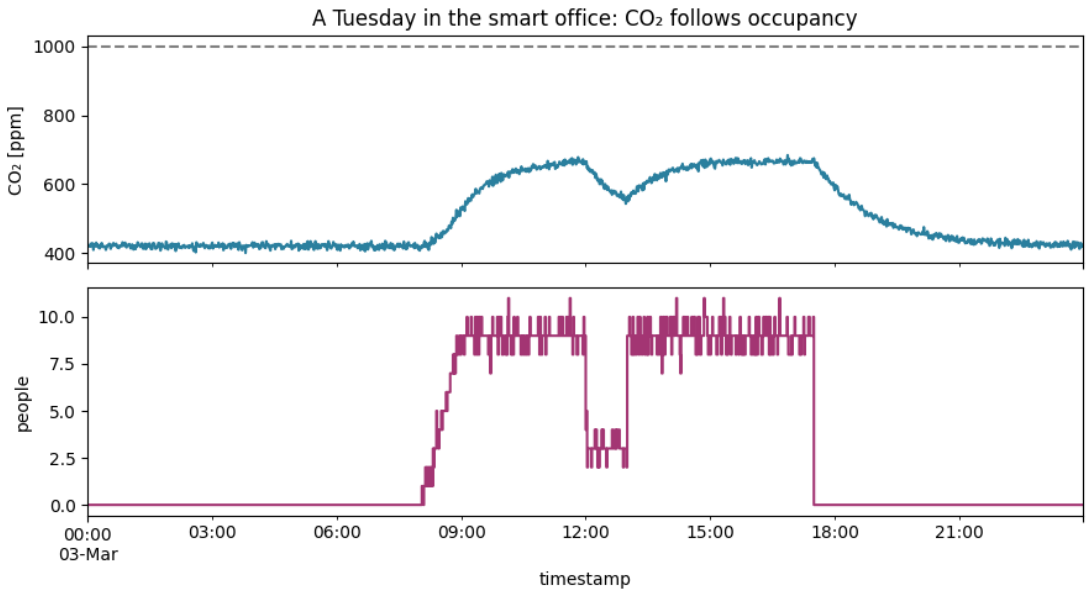


Figura 3. O zi de marți în biroul inteligent: CO_2 (sus) urmărește ocuparea (jos). Realizată în notebook-ul cu exemple rezolvate.

3 Miniproiect: raportul privind calitatea aerului din birou

Combinăm tot ce am învățat: **fișier → buclă → funcție → clasă → raport**. Programul citește coloana CO_2, respinge valorile imposibile fizic cu ajutorul clasei `Co2Sensor` și sintetizează calitatea aerului.

```
def load_co2(path):
    """Returnează o listă de perechi (timestamp, co2), omițând valorile lipsă."""
    out = []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            try:
                out.append((row["timestamp"], float(row["co2_ppm"])))
            except ValueError:
                continue
    return out

def classify(value):
    return "bun" if value < 600 else "acceptabil" if value < 1000 else "slab"

data = load_co2("smart_office_dataset.csv")
sensor = Co2Sensor("co2-lab204")
rejected = [ts for ts, v in data if not sensor.add(v)]
levels = Counter(classify(v) for v in sensor.readings)
total = sum(levels.values())

print(sensor)
print("respinse ca imposibile:", len(rejected))
for level in ("bun", "acceptabil", "slab"):
    share = levels[level] / total
    bar = "█" * int(share * 40)
    print(f"{level:>10}: {levels[level]:5d} măsurători ({share:6.1%}) {bar}")
print("cea mai mare valoare:", max(sensor.readings), "ppm")
```

REZULTAT

```
Sensor('co2-lab204', 10047 valori, unit=ppm)
respinse ca imposibile: 2
       bun:  7496 măsurători ( 74.6%) ██████████████████████████████
acceptabil:  2547 măsurători ( 25.4%) ██████████
      slab:     4 măsurători (  0.0%)
cea mai mare valoare: 2400.0 ppm
```

PUNEȚI-VĂ REZULTATELE SUB SEMNUL ÎNTREBĂRII

Doar **4** măsurători din aproximativ 10 000 sunt „slabe”, toate între 2 100 și 2 400 ppm, în timp ce următoarea valoare ca mărime este de numai 746 ppm. Patru vârfuri izolate, de trei ori mai mari decât orice altceva, într-un birou care nu depășește niciodată circa 700 ppm, par **defecțiuni ale senzorului, nu aer prost**. Un raport care spune „în birou au fost 4 măsurători cu aer slab” ar induce în eroare. A învăța să vă *îndoiți* de un asemenea număr este ceea ce urmăresc cursurile de înțelegere a datelor (Data Understanding).

4 Concluzii și fișă recapitulativă

Ați parcurs minimul de Python necesar pentru laboratoarele următoare: bazele limbajului, mediul interactiv Jupyter și primul contact cu date reale de la senzori. Tutorialul este scurt în mod intenționat; resursele de la finalul îndrumarului vă duc mai departe.

Temă	Rețineți
Tipuri	totul este un obiect; tipizare dinamică, dar puternică; comparați numerele `float` cu `math.isclose`
Containere	`list`, `tuple`, `set`, `dict` (ordonat după inserare din versiunea 3.7); secționare `a[start:stop:step]`; comprehensiuni
Șiruri	folosiți f-string-uri: `f"{x:.2f}"`
Structuri de control	`range` exclude capătul; `break`, `continue`, `for … else`
Funcții	docstring, valori implicite, argumente cu nume, mai multe valori returnate, `lambda`
Clase	`__init__`, `self`, moștenire, `__repr__`, `@dataclass`
Erori	`try / except EroareSpecifică`; a intercepta o eroare nu înseamnă a valida datele
Fișiere	`with open(...)`, `pathlib`, `csv`, `json`
Instrumente	mediu virtual, `%pip install`, Colab / VS Code / JupyterLab

Lecturi suplimentare

- Tutorialul Python și documentația Python 3
- PEP 8 – Ghid de stil pentru codul Python
- GeeksforGeeks – Python programming tutorial
- Codecademy – Learn Python
- Mark Lutz, *Learning Python*, O'Reilly · Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, O'Reilly · François Chollet, *Deep Learning with Python*, Manning

5 Temă

A Partea A – Pregătiți-vă mediul de lucru

- Configurați-vă mediul Python ca în secțiunea 1.2. Folosiți un **mediu virtual** pe calculatorul vostru sau Colab.
- Instalați Jupyter (Notebook și JupyterLab) și porniți-l.
- Creați un notebook nou și afișați `Hello world!`.
- Deschideți `Lab1_Exemple_Rezolvate.ipynb` și rulați fiecare celulă. Rezolvați sarcinile **Încercați**.
- Rulați celula de importuri de mai jos. Fiecare linie trebuie să funcționeze. (Pachetele `altair`, `bokeh`, `holoviews`, `datashader`, `wordcloud` și `spacy` din edițiile anterioare **nu** sunt necesare la acest curs.)

PRIMA CELULĂ A NOTEBOOK-ULUI VOSTRU

```
%pip install numpy scipy pandas matplotlib seaborn scikit-learn statsmodels
```

A DOUA CELULĂ

```
import numpy, scipy, pandas, matplotlib, seaborn, sklearn, statsmodels
print("numpy", numpy.__version__, "| pandas", pandas.__version__, "| seaborn",
seaborn.__version__)
```

B Partea B – Exerciții

Deschideți **`Lab1_Exercitii.ipynb`**. Conține opt probleme pe datele biroului inteligent. Fiecare problemă are un enunț, o celulă de cod cu `...` pentru rezolvarea voastră și o **autoverificare** care afișează ✅ sau ❌ împreună cu un indiciu.

Nr.	Tema	Nivel
1	Variabile și operatori: conversii de temperatură	★☆☆
2	Liste, secționare și comprehensiuni	★☆☆
3	Bucle și decizii: controlerul de ventilație	★★☆
4	Funcții: clasificarea CO_2, sintetizarea măsurătorilor	★★☆
5	f-string-uri: panoul de stare al camerei de control	★★☆
6	Clase: modelați o cameră	★★☆
7	Fișiere și excepții: curățați un jurnal dezordonat	★★★
8	Miniproiect: când este biroul cel mai aglomerat?	★★★

CE PREDAȚI

Notebook-ul `Lab1_Exercitii.ipynb` după *Kernel → Restart & Run All*, cu toate autoverificările în verde și cu răspunsurile scrise completate. Încărcați-l pe platforma cursului.